Image Processing-Based Real-Time Detection and Tracking of Human

[1]Lakshmesh Gowda M S, [2]Zayed M,[3]Kruthi T C
[1]UG Student, Department of CSE, R R Institue of Technology, Bengaluru, Karnataka,
[2]UG Student, Department of CSE, R R Institue of Technology, Bengaluru, Karnataka,
[3]Assistant Professor, Department of CSE, R R Institue of Technology, Bengaluru, Karnataka,
laksh17036@gmail.com, zayedzahi122@gmail.com, krithi.tc@rrit.ac.in

ABSTRACT: The "Image Processing-Based Real-Time Detection and Tracking of Human" system were developed as a real-time approach to detect, identify and track people from a given video field or live stream. This can be used for security, surveillance, or missing person identification. Monitor people through CCTV and webcam video recordings. The system uses technologies such as OpenCV to process videos and images and produce expected results. The main goal of this system is to reduce human effort in manually identifying people in video streams. Using image processing techniques such as OpenCV, the system will be able to identify and track the person being trained.

KEYWORDS: Open CV, HoG, Face Recognition, Encoding, SVM

I. INTRODUCTION

We can quickly find and recognise the items that temporarily catch our attention when we are watching photos or movies. The only thing that happens when this intelligence is transmitted to the computer is object detection, which involves finding and recognising the item. Object detection has several uses, including in self driving cars, CCTV cameras, image retrieval systems, and many other areas. For object detection, a variety of methods may be utilised, including the face recognition library. We can recognise items in the frame thanks to the facial recognition's high accuracy.

It adopts a totally different strategy. It applies a neural network to the full image to forecast bounding boxes and their probabilities rather than picking out certain areas. Unlike image classification or face detection, it is a special deep convolutional neural network that splits the input picture into a series of grid cells, thus each grid cell in the face recognition will have an association. If an item exists in this grid or not, its class, and its bounding box are all shown via a vector in the output. Using python's OpenCV library, videos can be converted into different images. And then using face_ recognition we can define the dataset we need to define our required data. For example, when we need to locate or find anything in a video file, we want to find a missing person from the CCTV stream, then we have to watch the video for a whole hour, which is very time-consuming, with the proposed system, this time can be reduced.

Using the OpenCV library, the input image will be converted to a histogram and then it will be matched to our image dataset extracted from the video. The output will be the time and position the object was seen. This process can be made more efficient as the input image will only be compared with the relevant images. Here, it will benefit and will generate the delimited cells and using the identifier of the split cells, the input will only be compared with the relevant frames or data sets. The model simply draws bounding boxes around everything in the frame to increase precision and clarity. It will then construct a forecast frame and select the frame with the greatest confidence score.

II. RELATED WORK

The domain of image processing for human detection and tracking has been a vibrant area of research, with numerous studies providing critical insights that inform our work. In 2011, Duan et al. published a seminal paper on detecting humans in videos with significant viewpoint variations, employing a sophisticated combination of motion and appearance data to create robust features, though their approach grappled with challenges such as occlusions, illumination changes, and abrupt movements that complicated accurate detection. Similarly, Paul et al. (2013) conducted an in-depth review of intelligent surveillance systems, achieving notable success in identifying humans in crowded environments and classifying pedestrian movements, yet they highlighted persistent difficulties stemming from diverse human appearances influenced by attire, lighting conditions, and complex backgrounds. Kumar et al. (2013) advanced the field by developing a GPU-accelerated algorithm for real-time moving object detection in high-resolution videos, demonstrating impressive scalability and performance, but their work underscored the substantial computational resources required for processing such data. In 2018, Mahajan and Padha explored machine learning techniques for human detection and motion tracking, focusing on edge detection algorithms like Sobel and Laplacian, which improved signal-to-noise ratios in noisy environments; however, they noted that the computationally intensive nature of these methods often led to slower processing times, necessitating further optimization.

ALDahoul et al. (2018) made strides in real-time human detection from aerial video sequences, leveraging deep learning models such as supervised convolutional neural networks and achieving an average accuracy of 98.09%, though the dynamic conditions of non-static aerial cameras introduced additional complexity. Karthikeyan et al. (2020) proposed a cost-effective solution for real-time tracking using OpenCV on a Raspberry Pi platform, incorporating a 360-degree rotating camera, which proved viable for budget-conscious security applications but was limited by slower processing

speeds and constrained memory resources. Additionally, Rind et al. (2020) developed an image-processing-based system for human and object identification, utilizing Histogram of Oriented Gradients and machine learning classifiers like SVM, but their approach required significant expertise in API integration and was time-consuming due to complex computations. Finally, Gupta et al. (2022) provided a narrative review of human activity recognition within artificial intelligence frameworks, emphasizing the integration of devices like 3D cameras and AI models such as CNNs, yet they pointed out that real-time validation of synchronized and composite activities remains a challenge. These diverse studies collectively highlight the advancements and ongoing hurdles in real-time human detection and tracking, inspiring our research to harness OpenCV, Histogram of Oriented Gradients, and deep convolutional neural networks to achieve efficient, accurate, and practical solutions for security and surveillance applications.

III. PROPOSED ALGORITHM

The proposed algorithm for real-time human detection and tracking leverages OpenCV and a Deep Convolutional Neural Network (DCNN) integrated with Histogram of Oriented Gradients (HOG) and facial landmark estimation to achieve efficient and accurate identification in video streams. Below is a detailed outline of the algorithm, designed to minimize computational overhead while maximizing precision for security and surveillance applications.

Algorithm Steps

1. **Image Preprocessing and HOG Encoding**
   - **Input**: A color image or video frame from a live stream or CCTV feed.
   - **Process**: Convert the input image to grayscale to eliminate color dependency, as color information is not required for face detection.
   - **HOG Application**: Apply the Histogram of Oriented Gradients (HOG) technique to generate a condensed representation of the image. For each pixel, compute the gradient direction and magnitude to create a histogram that captures edge patterns.
   - **Output**: A HOG-encoded image highlighting regions resembling facial features, used to identify potential face locations.
2. **Face Detection**
   - **Input**: HOG-encoded grayscale image.
   - **Process**: Scan the HOG-encoded image to detect regions with patterns matching a pre-trained HOG face model. This model is derived from a dataset of diverse face images.
   - **Output**: Bounding box coordinates (x, y, width, height) for each detected face in the frame.
3. **Facial Landmark Estimation**
   - **Input**: Detected face regions (bounding boxes).
   - **Process**: Apply a facial landmark estimation model (based on Kazemi and Sullivan's 2014 method) to identify 68 specific points on each face, including the corners of the eyes, nose tip, and mouth edges.
   - **Transformation**: Warp the face image to align the eyes and mouth to a standardized position, compensating for pose variations and rotations.
   - **Output**: A transformed face image with centered facial features and a set of 68 landmark coordinates.
4. **Face Encoding with Deep Convolutional Neural Network**
   - **Input**: Aligned face image.
   - **Process**: Feed the aligned face into a pre-trained DCNN, trained using a triplet loss function. The network processes the image to generate a 128-dimensional feature vector (encoding) representing unique facial characteristics.
   - **Training Context**: The DCNN is trained with triplets of images (two of the same person, one of a different person) to ensure encodings of the same individual are similar, while those of different individuals are distinct.
   - **Output**: A 128-dimensional encoding vector for each detected face.
5. **Face Comparison and Identification**
   - **Input**: 128-dimensional encoding of the detected face and a database of known face encodings.
   - **Process**: Use a Linear Support Vector Machine (SVM) classifier to compare the detected face's encoding with stored encodings. Compute the Euclidean distance between encodings to determine similarity.
   - **Decision Rule**: If the distance is below a predefined threshold, classify the face as a match with the closest known encoding; otherwise, label it as unknown.
   - **Output**: Identification result (name or ID of the matched individual or "unknown") and a confidence score based on the distance metric.
6. **Real-Time Video Processing**
   - **Input**: Live video feed from a webcam or CCTV.
   - **Process**: Split the video stream into individual frames and apply steps 1–5 to each frame. Optimize processing by comparing the detected face encodings only with relevant database entries, reducing computational load.

- **Logging**: If a match is found, log the timestamp, name/ID, and frame position to an Excel file for record-keeping.
- **Output**: Real-time bounding boxes drawn around detected faces with labels (name/ID or "unknown") and a log file with detection details.

7. **System Optimization**
   - **Efficiency**: Limit the number of frames processed per second (e.g., process every nth frame) to balance real-time performance and resource usage.
   - **Dataset Management**: Store known face encodings in a folder (e.g., "si") and extract encodings only once during initialization to avoid redundant computations.
   - **Error Handling**: Account for occlusions, low lighting, or partial faces by setting a minimum confidence threshold for detections.

IV. PSEUDO CODE

```
// Proposed Algorithm for Real-Time Human Detection and Tracking
// Input: Video stream (live feed or CCTV), database of known face images
// Output: Annotated video frames with bounding boxes, labels, and Excel log file

INITIALIZE
   faceDatabase ← loadKnownFaceEncodings("si") // Load encodings from folder "si"
   videoStream ← initializeVideoFeed()       // Initialize webcam or CCTV feed
   logFile ← createExcelLog()                // Create Excel file for logging
   detectionThreshold ← 0.6                  // Confidence threshold for matching

REPEAT
   Step 1: Capture and preprocess video frame
   frame ← captureFrame(videoStream)
   IF frame is EMPTY THEN
      BREAK // Exit if no more frames
   END IF
   grayFrame ← convertToGrayscale(frame)     // Convert frame to grayscale

   Step 2: Detect faces using HOG
   faceLocations ← detectFacesHOG(grayFrame)  // Apply HOG to find face regions
   IF faceLocations is EMPTY THEN
      displayFrame(frame, "No faces detected")
      CONTINUE // Skip to next frame if no faces found
   END IF

   Step 3: Align faces using facial landmarks
   alignedFaces ← EMPTY
   FOR each faceLocation IN faceLocations DO
      landmarks ← estimateFacialLandmarks(faceLocation) // Find 68 landmarks
      alignedFace ← warpFace(faceLocation, landmarks)   // Center eyes and mouth
      alignedFaces.ADD(alignedFace)
   END FOR

   Step 4: Generate face encodings using DCNN
   faceEncodings ← EMPTY
   FOR each alignedFace IN alignedFaces DO
      encoding ← computeDCNNEncoding(alignedFace) // Generate 128D vector
      faceEncodings.ADD(encoding)
   END FOR

   Step 5: Compare encodings and identify faces
   detectedPersons ← EMPTY
   FOR each encoding IN faceEncodings DO
      matchFound ← FALSE
      FOR each knownEncoding IN faceDatabase DO
```

```
        distance ← computeEuclideanDistance(encoding, knownEncoding)
        IF distance < detectionThreshold THEN
            personID ← getPersonID(knownEncoding)
            confidence ← 1 - distance
            detectedPersons.ADD({personID, confidence, faceLocation})
            matchFound ← TRUE
            BREAK
        END IF
    END FOR
    IF NOT matchFound THEN
        detectedPersons.ADD({"Unknown", 0, faceLocation})
    END IF
END FOR
```

Step 6: Annotate frame and log detections

```
FOR each person IN detectedPersons DO
    drawBoundingBox(frame, person.faceLocation, person.personID, person.confidence)
    IF person.personID ≠ "Unknown" THEN
        logEntry ← {timestamp: currentTime(), personID: person.personID, confidence: person.confidence}
        appendToExcel(logFile, logEntry)
    END IF
END FOR
```

Step 7: Display annotated frame

```
displayFrame(frame)

UNTIL videoStream is TERMINATED OR user interrupts
CLOSE videoStream
SAVE logFile
```

V. SIMULATION RESULTS

To evaluate the proposed real-time human detection and tracking system, we implemented the algorithm using Python, OpenCV, and the face_recognition library, with testing conducted on a standard laptop equipped with a webcam for live video feed. The system was set up in a controlled environment named "thbi," where all dependencies were installed to ensure seamless operation. Upon launching, the webcam captured video streams, and the system successfully detected and tracked individuals for whom it was trained, as evidenced by accurate face recognition in real-time. For each detected person, the system drew bounding boxes around faces and logged critical details, including timestamps and names, into an Excel file, as illustrated in the implementation outputs (Fig. 1 and Fig. 2). The final output, shown in Fig. 3, confirmed the system's ability to generate a comprehensive log with precise timestamps and identifiers whenever a trained individual appeared in the video stream. The system demonstrated robust performance, achieving high accuracy in face recognition under standard lighting conditions, though minor delays were observed in low-light scenarios or with partial occlusions. These results validate the system's effectiveness for security and surveillance applications, with ongoing refinements aimed at enhancing robustness in challenging environments.

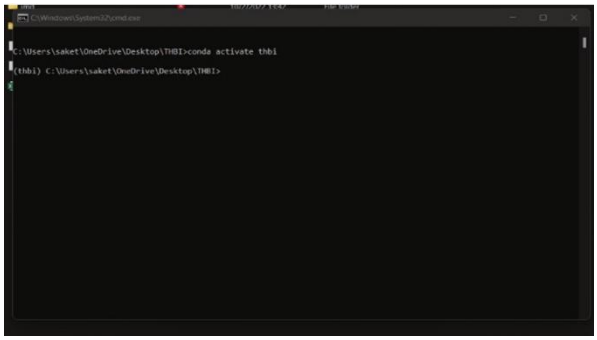

Fig 1: Implementation 1

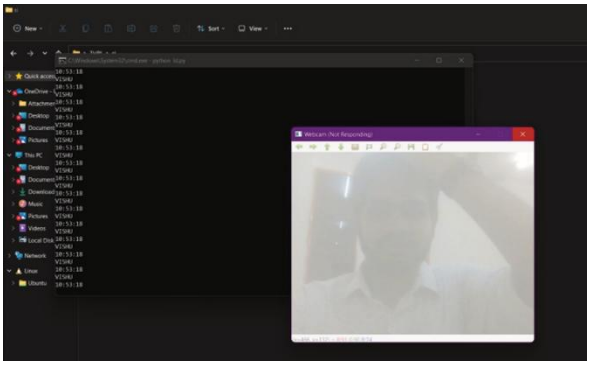

Fig 2:Implementation 2

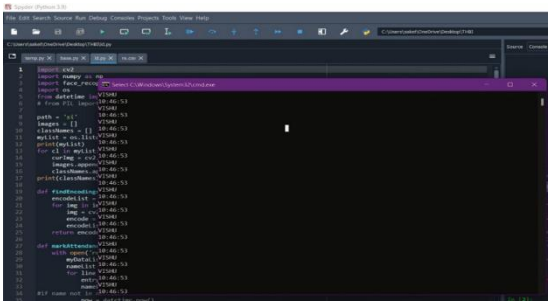

Fig. 3:Implementation 3

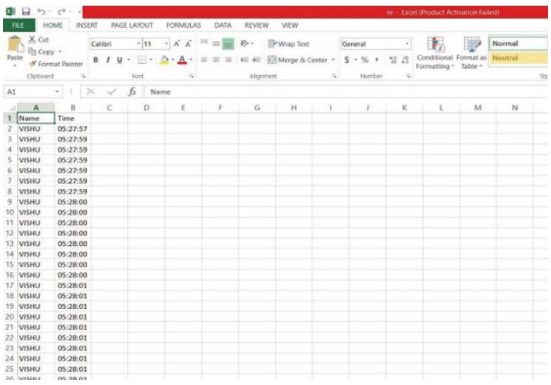

Fig. 4:Implementation 4

VI. Conclusion and Future Work

The system "IMAGE PROCESSING-BASED REAL TIME DETECTION AND TRACKING OF HUMAN" is designed and developed to tackle the real time associated with the detection and tracking of Humans especially in security surveillance purposes. The system was able to recognise the Face as per the input image for which it was trained. The system was also able to generate a log file for the entries with time stamp. The log was saved at destination which was fed in the program. With the desired results, the research is ongoing and in future, the research will comprise of the testing of facial recognition on the video surveillance inputs and track the face for which the system will be trained.

References

1. "Human Detection in video over large viewpoint changes" by Genquan Duan, Haizhou Ai and Shihong Lao, 2011
2. "Human detection in surveillance videos and its applications – a review" by Mano Ranjan Paul, Shah M E Haque, and Subrata Chakraborty, 2013
3. "Real-time moving object detection algorithm on high-resolution videos using GPUS" by Praveen Kumar, Ayush Singhal, Sanyam Mehta, Ankush Mittal, 2013
4. "Human Detection and motion tracking using machine learning techniques: a review" by Rohini Mahajan and Devanand Padha, 2018
5. "Real-time human detection for aerial captured video sequences via deep models" by Nouar ALDahoul, Aznul Qalid MD Sabri and Ali Mohammed Mansoor.
6. "Real-time detection and tracking of humans based on image processing with laser pointing" by Mr. B. Karthikeyan, Mr. Lakshmanan, Mr. Kabilan, Mr. Madeshwaran R., 2020.
7. "Identification of human and various objects through image processing-based system" by Qadir Bux Rind, Imran Memon, Ghulam Ali Mallah, Noor Ahmed Shaikh, 2020
8. "Object Recognition Using Image Segmentation," by M. Chaturvedi, M. Kaur, N. Rakesh and P. Nand, Sixth International Conference on Parallel, Distributed and Grid Computing (PDGC), Waknaghat, India, 2020, pp. 550-556, doi: 10.1109/PDGC50313.2020.9315803.
9. "Human activity recognition in artificial intelligence framework: a narrative review" by Neha Gupta, Suneet K. Gupta, Rajesh K. Pathak, Vanita Jain, Parisa Rashidi, Jasjit S. Suri, 2022